
Prof. Ryan Cotterell, Prof. Mrinmaya Sachan, Prof. Florian Tramèr

Assignment 1

Rasim Ibadov, 19-946-714

Collaborators

1. Rasim Ibadov, 19-946-714

Declaration

I, Rasim Ibadov, confirm that I have written this assignment independently, without any direct copying or paraphrasing from other students' work. I have acknowledged everyone I consulted during my research. I understand that while consultation and comparison of solutions are permitted, the final written submission must be my own work.

I am aware that any form of plagiarism is a violation of academic integrity. If it is determined that I have engaged in academic dishonesty, I accept and understand that the university's standard measures for plagiarism will be applied.

By submitting this assignment, I affirm my commitment to academic honesty and integrity.

Question 1: Representation Surgery

- a) *Proof.* No. Linear guardedness only rules out improvements by affine predictors. A general classifier may still exploit nonlinear information in X and achieve a loss below the trivially attainable loss L_τ . ■
- b) *Proof.* Let $f(X) = AX + b$ be any affine predictor and write the cross-entropy loss as $\mathcal{L}(\mathbf{y}, Z = i) = -y_i + \ln \sum_{j=1}^k e^{y_j}$. The function $\mathbf{y} \mapsto \mathcal{L}(\mathbf{y}, Z = i)$ is convex, hence $A \mapsto \mathbb{E}[\mathcal{L}(AX + b, Z)]$ is convex as well. Assume that

$$\mathbb{E}[X \mid Z = i] = \mathbb{E}[X]$$

for every i . Then, for any affine predictor and for an optimal constant predictor b^* , the first-order variation around the constant predictor is governed by the class-conditional means. In particular, at $A = 0$ the gradient $\nabla_A \mathbb{E}[\mathcal{L}(AX + b, Z)] = \mathbb{E}[(p(AX + b) - e_Z)X^\top] = \mathbb{E}[(p(b) - e_Z)X^\top] = p(b)\mathbb{E}[X^\top] - \mathbb{E}[e_Z X^\top] = p(b)\mathbb{E}[X^\top] - \sum_{i=1}^k \mathbb{P}[Z = i]e_i \mathbb{E}[X^\top \mid Z = i] = p(b)\mathbb{E}[X^\top] - \sum_{i=1}^k \mathbb{P}[Z = i]e_i \mathbb{E}[X^\top] = (p(b) - p(b^*))\mathbb{E}[X^\top] = 0$ in the affine directions, with one-hot label e_i , vanishes because the same mean $\mathbb{E}[X]$ appears in every class. Since the objective is convex, any stationary point is globally optimal, and thus no affine predictor can improve upon the best constant predictor. Hence X linearly guards Z . ■

- c) *Proof.* Let

$$p_j(\mathbf{y}) = \frac{e^{y_j}}{\sum_{m=1}^k e^{y_m}}.$$

Then

$$\frac{\partial}{\partial y_r} \mathcal{L}(\mathbf{y}, Z = i) = p_r(\mathbf{y}) - \mathbf{1}_{r=i}.$$

Since $0 \leq p_r(\mathbf{y}) \leq 1$, all partial derivatives are bounded in absolute value by 1. If $j_1, j_2 \neq i$, then

$$\frac{\partial}{\partial y_i} \mathcal{L}(y, Z = j_1) = p_i(y) = \frac{\partial}{\partial y_i} \mathcal{L}(y, Z = j_2),$$

and this quantity is strictly positive because $p_i(\mathbf{y}) > 0$ for finite $\mathbf{y}$. ■

- d) *Proof.* Let $\ell(A, b) = \mathbb{E}[\mathcal{L}(AX + b, Z)]$. If the infimum among constant predictors is attained at b^* and X linearly guards Z , then $(A, b) = (0, b^*)$ is optimal. The first-order condition in A is

$$0 = \nabla_A \ell(A, b) \big|_{A=0, b=b^*} = \mathbb{E}[(p(b^*) - \mathbf{1}_{Z=i})X^\top],$$

where $p(b^*) = \text{softmax}(b^*)$. By 1. c), the derivatives of the cross-entropy are bounded, so the dominated convergence theorem justifies exchanging differentiation and expectation. Taking the i -th class component and using $\mathbb{P}[Z = i] > 0$ for all i gives

$$\mathbb{E}[(p_i(b^*) - \mathbf{1}_{Z=i})X] = p_i(b^*)\mathbb{E}[X] - \mathbb{E}[X\mathbf{1}_{Z=i}] = 0.$$

Because b^* is the optimal constant predictor: $p_i(b^*) = \mathbb{P}[Z = i]$. Hence

$$\mathbb{E}[X\mathbf{1}_{Z=i}] = \mathbb{P}[Z = i]\mathbb{E}[X].$$

Dividing by $\mathbb{P}[Z = i] > 0$ gives $\mathbb{E}[X | Z = i] = \mathbb{E}[X]$ for all i . ■

e) *Proof.* For the one-hot vector $\tilde{Z}$, its i -th component is $\tilde{Z}_i = \mathbf{1}_{Z=i}$. Hence

$$\text{Cov}(X, \tilde{Z}_i) = \mathbb{E}[X\tilde{Z}_i] - \mathbb{E}[X]\mathbb{E}[\tilde{Z}_i] = \mathbb{P}[Z = i](\mathbb{E}[X | Z = i] - \mathbb{E}[X]).$$

Since $\mathbb{P}[Z = i] > 0$ for all i , we have

$$\text{Cov}(X, \tilde{Z}_i) = 0 \iff \mathbb{E}[X | Z = i] = \mathbb{E}[X].$$

$\Sigma_{X, \tilde{Z}}$ has columns $\text{Cov}(X, \tilde{Z}_i)$, hence

$$\Sigma_{X, \tilde{Z}} = 0 \iff \text{Cov}(X, \tilde{Z}_i) = 0 \forall i.$$

Therefore,

$$\mathbb{E}[X | Z = i] = \mathbb{E}[X] \forall i \iff \Sigma_{X, \tilde{Z}} = 0. \quad \text{■}$$

f) *Proof.* For $r(X) = PX + b$, linearity of covariance gives

$$\text{Cov}(r(X), \tilde{Z}) = \text{Cov}(PX + b, \tilde{Z}) = P\text{Cov}(X, \tilde{Z}) = P\Sigma_{X, \tilde{Z}}.$$

By the previous part, $r(X)$ linearly guards Z if and only if $\text{Cov}(r(X), \tilde{Z}) = 0$. Therefore $r(X)$ linearly guards $Z \iff P\Sigma_{X, \tilde{Z}} = 0$. ■

g) *Proof.* Assume first that $\mathbb{E}[X] = 0$. Then

$$\begin{aligned} D(P, b) &= \mathbb{E}\|PX + b - X\|^2 = \mathbb{E}\|(P - I)X + b\|^2 \\ &= \mathbb{E}\|(P - I)X\|^2 + 2\mathbb{E}\langle (P - I)X, b \rangle + \|b\|^2. \end{aligned}$$

Since $\mathbb{E}[X] = 0$, the cross term vanishes, so

$$D(P, b) = \mathbb{E}\|PX - X\|^2 + \|b\|^2.$$

In particular, the term $\|b\|^2$ is independent of P and minimized at $b = 0$. Hence minimizing $D(P, b)$ over (P, b) is equivalent to minimizing $D(P) = \mathbb{E}\|PX - X\|^2$. ■

h) *Proof.* Let $A = \Sigma_{X,\tilde{Z}}$ and assume $\mathbb{E}[X] = 0$ and $\Sigma_{X,X} = I$. Define

$$P^* = I - AA^+.$$

Then

$$P^*A = (I - AA^+)A = A - AA^+A = 0,$$

so the guarding constraint is satisfied. We write distortion as

$$D(P) = \mathbb{E}\|(P - I)X\|^2 = \text{tr}((P - I)\Sigma_{X,X}(P - I)^\top) = \|P - I\|_F^2.$$

Under the constraint $PA = 0$, the matrix P must vanish on $\text{colsp}(A)$, while on $\text{colsp}(A)^\perp$ the closest map to the identity is the orthogonal projection onto that orthogonal complement. By the projection theorem, this projection is exactly $I - AA^+$. Hence P^* minimizes the distortion among all feasible matrices. ■

i) *Proof.* Let $\mu = \mathbb{E}[X]$ and $\Sigma = \Sigma_{X,X}$. We define the whitening matrix

$$W = (\Sigma^{1/2})^+,$$

so that $\hat{X} = W(X - \mu)$ has identity covariance on the support of X . In whitened coordinates, the optimal projection is

$$\Pi_{W\Sigma_{X,\tilde{Z}}} = (W\Sigma_{X,\tilde{Z}})(W\Sigma_{X,\tilde{Z}})^+.$$

Therefore we first center, then whiten, project away the concept subspace, and finally unwhiten and add the mean back. This yields

$$r(x) = x - W^+ \Pi_{W\Sigma_{X,\tilde{Z}}} W(x - \mu).$$

This is exactly the stated full transformation. ■

j) See pages attached. Please note that these can't be shortened as they were given as template.

Question 2: Properties of Finite-state Language Models

Let $n = |Q|$ and $T = \sum_{y \in \Sigma} T(y)$. Let $\mathbf{1}$ denote the all-ones vector.

a) *Proof.* For each state q , let m_q be the expected remaining string length starting from q . Conditioning on the first transition gives

$$m = T(\mathbf{1} + m),$$

so $(I - T)m = T\mathbf{1}$. Since all eigenvalues of T have magnitude < 1 , the matrix $I - T$ is invertible and

$$m = (I - T)^{-1}T\mathbf{1}.$$

Finally, the expected length from the initial distribution is

$$\mu = \lambda^\top m.$$

The computation requires solving one linear system with an $n \times n$ matrix, which can be done in $O(n^3)$ time. ■

b) *Proof.* Let

$$s_q = \mathbb{E}[|Y|^2 \mid q].$$

Again conditioning on the first transition yields

$$s = T(\mathbf{1} + 2m + s).$$

Hence $(I - T)s = T(\mathbf{1} + 2m)$. After solving for s , we get

$$\mathbb{E}[|Y|^2] = \lambda^\top s, \sigma^2 = \lambda^\top s - \mu^2.$$

This is one linear solve, so runtime is $O(n^3)$. ■

c) *Proof.* In the deterministic case, each state has at most one outgoing transition for a given symbol. Let H_q be the entropy of the string distribution generated from state q . Conditioning on the first emitted transition or termination gives

$$H_q = \xi_q + \sum_{q'} T_{q,q'} H_{q'},$$

where

$$\xi_q = - \sum_{y/w: q \xrightarrow{y/w} q'} w \log w - \rho_q \log \rho_q.$$

In vector form, $H = \xi + TH$, so $H = (I - T)^{-1}\xi$. The entropy of the whole language is then

$$H(p_{LM}) = \lambda^\top H = \lambda^\top (I - T)^{-1}\xi.$$

■

Question 3: Efficiently Simulating Bounded Stacks with RNNs

a) *Proof.* For $D(2, 3)$, reading $\rangle i$ pops the top symbol if it equals i and otherwise rejects. Reading $\langle i$ pushes i if the depth is still below 3, and rejects if the stack is already full. Applying these rules to the three given stacks gives the configurations listed in Table 1. ■

	$\gamma_1 = \langle_2$	$\gamma_2 = \langle_1 \langle_1 \langle_1$	$\gamma_3 = \langle_2 \langle_1 \langle_2$
$\rangle_2$	$\emptyset$	reject	$\langle_2 \langle_1$
$\rangle_1$	reject	$\langle_1 \langle_1$	reject
$\langle_2$	$\langle_2 \langle_2$	reject	reject

Table 1: The first row of the table represents different stack configurations, while the first column lists the newly read symbols.

b) *Proof.* Let the alphabet be

$$\Sigma = \{\langle 1, \dots, \langle k, \rangle 1, \dots, \rangle k\}.$$

The states are all stacks of depth at most m , together with a rejecting sink state R :

$$Q = \{\gamma : \gamma \text{ is a stack over } \{1, \dots, k\}, |\gamma| \leq m\} \cup \{R\}.$$

The initial state is the empty stack $\emptyset$, and the only accepting state is also $\emptyset$. The transition function is defined by the stack semantics: push on an opening bracket, pop on a matching closing bracket, and go to R on any invalid move. Since every bounded stack configuration is represented by a distinct state, the automaton recognizes exactly $D(k, m)$. The number of stack states is $1 + k + k^2 + \dots + k^m = O(k^m)$, so the total number of states is $O(k^m)$, and the number of transitions is $O(k \cdot k^m) = O(k^{m+1})$. ■

c) *Proof.* The hidden-state encoding in Eq. (27) stores the stack symbols from top to bottom, one-hot per stack cell, followed by zero padding. Thus the updated stacks after reading each symbol are represented by the corresponding concatenation of basis vectors shown in Table 2. ■

d) *Proof.* Let $S_\downarrow$ denote the block-shift matrix that implements a push by shifting all stack blocks down by one position, and let $S_\uparrow$ denote the corresponding pop matrix. For an opening bracket $\langle i$, define $U(\langle i) = S_\downarrow$, $V o_{\langle i} = e_i$ in the top block and for a closing bracket $\rangle i$, define $U(\rangle i) = S_\uparrow$, $V o_{\rangle i} = 0$. The eos symbol acts as the identity update. By construction, multiplying the previous hidden state by the corresponding matrix exactly implements the push or pop operation on the stack encoding. Hence the RNN dynamics mirrors the automaton transitions. ■

	$\gamma_1 = \langle_2$	$\gamma_2 = \langle_1 \langle_1 \langle_1$	$\gamma_3 = \langle_2 \langle_1 \langle_2$
$\rangle_2$	$h(\emptyset) = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$	reject	$h(\langle_2 \langle_1) = \begin{pmatrix} 1 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}$
$\rangle_1$	reject	$h(\langle_1 \langle_1) = (1, 0, 1, 0, 0, 0)^\top$	reject
$\langle_2$	$h(\langle_2 \langle_2) = (0, 1, 0, 1, 0, 0)^\top$	reject	reject

Table 2: The first row of the table represents different stack configurations, while the first column lists the newly read symbols.

- e) *Proof.* To make the update independent of the current symbol, we duplicate the hidden space to compute both push and pop operations simultaneously. Let

$$h_{t-1} = (h^{(P)}, h^{(Q)})$$

where each half has dimension km . We define the recurrence matrix U in block form:

$$U = \begin{pmatrix} S_\downarrow & 0 \\ 0 & S_\uparrow \end{pmatrix},$$

where $S_\downarrow$ (first $km \times km$ block) shifts stack representations down (push operation) and $S_\uparrow$ (second $km \times km$ block) shifts them up (pop operation). The matrix $V \in \mathbb{R}^{2km \times (2k+1)}$ has the form

$$V = \begin{pmatrix} V_P \\ V_Q \end{pmatrix},$$

where $V_P, V_Q \in \mathbb{R}^{km \times (2k+1)}$ inject the input symbol's one-hot encoding into both the push and pop halves. Specifically, for each symbol $y \in \Sigma$, the columns $V o_y$ inject the symbol at the top of both stack blocks. The bias

$$b = (b^{(P)}, b^{(Q)}) \in \mathbb{R}^{2km}$$

encodes a threshold for each symbol type: opening brackets $\langle_i$ trigger positive bias in the first half (to select the push result), while closing brackets $\rangle_i$ trigger positive bias in the second half (to select the pop result). The Heaviside activation H applies element-wise, so

$$\begin{aligned} h_t^{(P)} &= H(S_\downarrow h_{t-1}^{(P)} + (V_P o_{y_t})^{(P)} + b^{(P)}), \\ h_t^{(Q)} &= H(S_\uparrow h_{t-1}^{(Q)} + (V_Q o_{y_t})^{(Q)} + b^{(Q)}). \end{aligned}$$

For an opening bracket $\langle i$, we set $b^{(P)} > 0$ and $b^{(Q)} < -\|S_1 h_{t-1}^{(Q)}\|$ so that H activates only the push half, and $h_t = (h_t^{(P)}, 0^{km})$. For a closing bracket $\rangle i$, we set $b^{(Q)} > 0$ and $b^{(P)} < 0$ so that H activates only the pop half, and $h_t = (0^{km}, h_t^{(Q)})$. For eos, set $b^{(\cdot)} \approx 0$ to activate both halves identically (identity update). By construction, h_t always equals either the push-updated or pop-updated stack representation depending on y_t , exactly mirroring the automaton's transition function. The output of the Heaviside function selects which half contains the valid next state. The overall dimensions are $U \in \mathbb{R}^{2km \times 2km}$, $V \in \mathbb{R}^{2km \times (2k+1)}$, $b \in \mathbb{R}^{2km}$. $\blacksquare$

f) *Proof.* Let $M = 2k + 1$ and let

$$\bar{\Sigma} = \Sigma \cup \{\text{eos}\}.$$

For a hidden state h encoding a valid bounded stack γ , define

$$A(h) \subseteq \bar{\Sigma}$$

as the set of acceptable one-step continuations under the stack automaton. Because the stack is encoded one-hot per position, the predicates "top symbol is i ", "stack is empty", and "depth is m " are linear threshold predicates in h . Hence for each $y \in \bar{\Sigma}$ there exists an affine score

$$s_y(h) = w_y^\top h + c_y$$

such that $s_y(h) > 0$ iff $y \in A(h)$ and $s_y(h) < 0$ iff $y \notin A(h)$. Collecting rows $w_y^\top$ gives $E \in \mathbb{R}^{M \times 2km}$ and collecting c_y gives $b' \in \mathbb{R}^M$.

Fix $\alpha > 0$ and scale/shift these affine scores so that for every valid stack-encoding state h ,

$$(Eh + b')_y = \begin{cases} +\alpha, & y \in A(h), \\ -\alpha, & y \notin A(h). \end{cases}$$

Then

$$p_{\text{SM}}(y \mid y_{<t}) = \text{softmax}(Eh_{t-1} + b')_y.$$

If $n_v = |A(h)|$, the normalizer is

$$Z(n_v) = n_v e^\alpha + (M - n_v) e^{-\alpha},$$

so

$$p_{\text{val}}(n_v) = \frac{e^\alpha}{Z(n_v)} = \frac{1}{n_v + (M - n_v) e^{-2\alpha}},$$

$$p_{\text{inv}}(n_v) = \frac{e^{-\alpha}}{Z(n_v)} = \frac{1}{n_v e^{2\alpha} + M - n_v}.$$

For valid stack states there are three structural cases: empty stack, non-empty non-full stack, and full stack; the corresponding valid-counts are $n_v \in \{k+1, k+1, 1\}$ (so two distinct normalizers). Therefore the worst-case valid probability is at $n_v = k+1$, and the worst-case invalid probability is at $n_v = 1$:

$$p_{\text{val}}(k+1) \geq \eta \iff e^{-2\alpha} \leq \frac{1/\eta - (k+1)}{k},$$

$$p_{\text{inv}}(1) < \eta \iff e^{2\alpha} > \frac{1}{\eta} - 2k.$$

Hence it suffices to choose

$$\alpha \geq \frac{1}{2} \log \frac{k}{1/\eta - (k+1)},$$

and additionally (only when $\eta < \frac{1}{2k}$, so that this condition is non-vacuous)

$$\alpha > \frac{1}{2} \log(1/\eta - 2k).$$

This is feasible in the regime of the statement $\eta < \frac{1}{k+1}$.

With this choice,

$$p_{\text{SM}}(y_t \mid y_{<t}) \geq \eta \iff y_t \in A(h_{t-1}),$$

so the locally η -truncated support equals exactly the set of strings that always take acceptable automaton transitions, i.e.

$$\mathcal{L}_\eta(p_{\text{SM}}) = D(k, m).$$

■

Let $\ell = \lceil \log k \rceil + 1$ and write $\mathbf{e}_i \in \{0, 1\}^\ell$ for the binary code of symbol i .

g) *Proof.* For $D(4, 3)$ we have $\ell = 3$ and the hidden state is the concatenation of the binary codes from top to bottom, followed by zero padding. Hence

$$\begin{aligned} h(\gamma_1) &= h(\langle_2) = (\mathbf{e}_2, 0, 0)^\top, \\ h(\gamma_2) &= h(\langle_1 \langle_3 \langle_1) = (\mathbf{e}_1, \mathbf{e}_3, \mathbf{e}_1)^\top, \\ h(\gamma_3) &= h(\langle_2 \langle_1 \langle_4) = (\mathbf{e}_4, \mathbf{e}_1, \mathbf{e}_2)^\top. \end{aligned}$$

Reading the requested inputs gives

	$\gamma_1 = \langle_2$	$\gamma_2 = \langle_1 \langle_3 \langle_1$	$\gamma_3 = \langle_2 \langle_1 \langle_4$
$\rangle_1$	reject	$h(\langle_1 \langle_3) = (\mathbf{e}_3, \mathbf{e}_1, 0)^\top$	reject
$\rangle_4$	reject	reject	$h(\langle_2 \langle_1) = (\mathbf{e}_1, \mathbf{e}_2, 0)^\top$
$\langle_3$	$h(\langle_3 \langle_2) = (\mathbf{e}_3, \mathbf{e}_2, 0)^\top$	reject	reject

■

- h) *Proof.* Relative to part (e), the hidden-state width changes from $2km$ to $2\ell m$, because each stack symbol is now stored using $\ell = \lceil \log k \rceil + 1$ bits instead of a k -dimensional one-hot vector. The block structure of the recurrence is unchanged: U is still a block shift matrix with a push half and a pop half, but each block is now $\ell m \times \ell m$. Likewise,

$$U \in \mathbb{R}^{2\ell m \times 2\ell m}, \quad V \in \mathbb{R}^{2\ell m \times (2k+1)}, \quad b \in \mathbb{R}^{2\ell m}.$$

The role of V is to inject the one-hot input symbol into the appropriate ℓ -bit code at the top of the stack, while b again selects the push or pop half. ■

- i) *Proof.* Let $A(h)$ be the set of acceptable continuations from hidden state h . As in part (f), we choose one affine score per symbol,

$$s_y(h) = w_y^\top h + c_y,$$

so that $s_y(h) > 0$ iff $y \in A(h)$ and $s_y(h) < 0$ otherwise. Since the predicates "top symbol is i ", "stack is empty", and "stack is full" are all threshold predicates in the binary code, such rows $w_y^\top$ exist. Stacking these rows gives

$$E \in \mathbb{R}^{(2k+1) \times 2\ell m}, \quad b' \in \mathbb{R}^{2k+1},$$

and after scaling by a sufficiently large α we may enforce

$$(Eh + b')_y = \begin{cases} +\alpha, & y \in A(h), \\ -\alpha, & y \notin A(h). \end{cases}$$

Then the same softmax argument as in part (f) shows that $\mathcal{L}_\eta(p_{\text{SM}}) = D(k, m)$. ■

Question 4: Limitations of Transformers on Simple Languages

- a) *Proof.* Since

$$z_t^{(\ell)} = \sum_{t'=1}^T s_{t,t'}^{(\ell)} v_{t'}^{(\ell)} = \left(\sum_{t'=1}^T s_{t,t'}^{(\ell)} x_{t'}^{(\ell)} \right) W_V^{(\ell)},$$

we can absorb the fixed linear map $W_V^{(\ell)}$ into the MLP. Define

$$f'(u, x) := f(u W_V^{(\ell)}, x).$$

Then

$$x_t^{(\ell+1)} = f\left(\sum_{t'=1}^T s_{t,t'}^{(\ell)} v_{t'}^{(\ell)}, x_t^{(\ell)}\right) = f'\left(\sum_{t'=1}^T s_{t,t'}^{(\ell)} x_{t'}^{(\ell)}, x_t^{(\ell)}\right).$$

So f' is simply the original MLP composed with the fixed value projection. ■

b) *Proof.* Write the attention output at layer ℓ and position t as

$$z_t^{(\ell)} = \sum_{t'=1}^T s_{t,t'}^{(\ell)} x_{t'}^{(\ell)}.$$

Since the softmax weights satisfy $s_{t,t'}^{(\ell)} \geq 0$ and $\sum_{t'=1}^T s_{t,t'}^{(\ell)} = 1$, this is a convex combination of the rows of $X^{(\ell)}$. Hence

$$\|z_t^{(\ell)}\| \leq \sum_{t'=1}^T s_{t,t'}^{(\ell)} \|x_{t'}^{(\ell)}\| \leq \|X^{(\ell)}\|_{2,\infty}.$$

Using $f(0,0) = 0$ and the L_f -Lipschitz property of f , we get

$$\|x_t^{(\ell+1)}\| = \|f(z_t^{(\ell)}, x_t^{(\ell)}) - f(0,0)\| \leq L_f(\|z_t^{(\ell)}\| + \|x_t^{(\ell)}\|) \leq 2L_f\|X^{(\ell)}\|_{2,\infty}.$$

Taking the maximum over t yields the recurrence

$$\|X^{(\ell+1)}\|_{2,\infty} \leq 2L_f\|X^{(\ell)}\|_{2,\infty}.$$

By induction on ℓ and the base case $\|X^{(0)}\|_{2,\infty}$, it follows that

$$\|X^{(\ell)}\|_{2,\infty} \leq (2L_f)^\ell \|X^{(0)}\|_{2,\infty}.$$

Equivalently, for every t ,

$$\|x_t^{(\ell)}\| \leq (2L_f)^\ell \|X^{(0)}\|_{2,\infty}.$$

■

c) *Proof.* Write the query and key vectors as

$$q_t^{(\ell)} = x_t^{(\ell)} W_Q^{(\ell)}, \quad k_{t'}^{(\ell)} = x_{t'}^{(\ell)} W_K^{(\ell)}.$$

Then the pre-softmax attention score is the bilinear form

$$l_{t,t'}^{(\ell)} = (q_t^{(\ell)})^\top k_{t'}^{(\ell)}.$$

By Cauchy–Schwarz and submultiplicativity of the operator norm,

$$|l_{t,t'}^{(\ell)}| \leq \|q_t^{(\ell)}\| \|k_{t'}^{(\ell)}\| \leq \|x_t^{(\ell)}\| \|W_Q^{(\ell)}\| \|x_{t'}^{(\ell)}\| \|W_K^{(\ell)}\|.$$

From part (b), every hidden state satisfies $\|x_t^{(\ell)}\| \leq F$, where

$$F \geq \max(1, (2L_f)^L) \|X^{(0)}\|_{2,\infty}.$$

Therefore

$$|l_{t,t'}^{(\ell)}| \leq \|W_Q^{(\ell)}\| \|W_K^{(\ell)}\| F^2.$$

Setting

$$L_{att} := \|W_Q^{(\ell)}\| \|W_K^{(\ell)}\|$$

gives the desired bound $\|\mathbf{l}\|_\infty \leq A$ with $A = F^2 L_{att}$.

■

d) *Proof.* Let $s = \text{softmax}(l)$ with $l \in \mathbb{R}^T$ and $\|l\|_\infty \leq A$. Then for each coordinate

$$s_t = \frac{e^{l_t}}{\sum_{j=1}^T e^{l_j}}.$$

Since $l_t \geq -A$ and $l_j \leq A$ for all j , we obtain

$$\frac{e^{-A}}{Te^A} \leq s_t \leq \frac{e^A}{Te^{-A}}.$$

Equivalently,

$$\frac{e^{-2A}}{T} \leq s_t \leq \frac{e^{2A}}{T}.$$

■

e) *Proof.* Add and subtract the mixed term $\sum_{t'=1}^T s'_{t,t'}^{(\ell)} x_{t'}^{(\ell)}$:

$$\begin{aligned} \Delta_t^{(\ell)} &= \sum_{t'=1}^T s_{t,t'}^{(\ell)} x_{t'}^{(\ell)} - \sum_{t'=1}^T s'_{t,t'}^{(\ell)} x_{t'}^{(\ell)} \\ &= \sum_{t'=1}^T (s_{t,t'}^{(\ell)} - s'_{t,t'}^{(\ell)}) x_{t'}^{(\ell)} + \sum_{t'=1}^T s'_{t,t'}^{(\ell)} (x_{t'}^{(\ell)} - x_{t'}'^{(\ell)}). \end{aligned}$$

The first sum (difficult part) captures changes in the softmax weights while keeping $x^{(\ell)}$ fixed. The second sum (easy part) captures changes in the hidden states while keeping the primed softmax weights $s'^{(\ell)}$ fixed. Taking norms and applying the triangle inequality gives exactly the stated decomposition into the difficult and easy parts. ■

f) *Proof.* The difficult part satisfies

$$\left\| \sum_{t'=1}^T (s_{t,t'}^{(\ell)} - s'_{t,t'}^{(\ell)}) x_{t'}^{(\ell)} \right\| \leq \sum_{t'=1}^T |s_{t,t'}^{(\ell)} - s'_{t,t'}^{(\ell)}| \|x_{t'}^{(\ell)}\|.$$

Since every row norm is at most $\|X^{(\ell)}\|_{2,\infty}$, the bound follows:

$$\leq \|X^{(\ell)}\|_{2,\infty} \sum_{t'=1}^T |s_{t,t'}^{(\ell)} - s'_{t,t'}^{(\ell)}|.$$

■

- g) *Proof.* Let f be twice differentiable and let (a_T) and (b_T) be bounded with $|a_T - b_T| = O(1/T)$. By the mean value theorem, for some ξ_T between a_T and b_T ,

$$|f(a_T) - f(b_T)| = |f'(\xi_T)| |a_T - b_T|.$$

Because the sequences are bounded and f' is continuous, $\sup_T |f'(\xi_T)| < \infty$, so the difference is $O(1/T)$. Applying the same argument to $f(x) = e^x$ yields

$$|e^{a_T} - e^{b_T}| = O(1/T).$$

■

- h) *Proof.* The softmax map is smooth on bounded sets. Its Jacobian has entries

$$\frac{\partial s_i}{\partial l_j} = s_i(\delta_{ij} - s_j).$$

Under the bounded-logit assumption, each $s_i = O(1/T)$ by part (d). Hence the Jacobian entries satisfy $\left| \frac{\partial s_i}{\partial l_j} \right| \leq |s_i| \leq O(1/T)$. By the mean value theorem applied to the softmax map,

$$|\text{softmax}(a_T)_t - \text{softmax}(b_T)_t| \leq \sup_c \|\nabla \text{softmax}(c)\| \cdot |a_T - b_T| = O(1/T) \cdot O(1/T) = O(1/T^2).$$

■

- i) *Proof.* Assume that all coordinatewise differences of the non-flipped positions satisfy $O(1/T)$. For $t' \neq t^*$, the differences in attention logits $|l_{t,t'}^{(\ell)} - l'_{t,t'}^{(\ell)}| = O(1/T)$ inherit from the hidden state differences. By part (h), this implies $|s_{t,t'}^{(\ell)} - s'_{t,t'}^{(\ell)}| = O(1/T^2)$ per term. Summing over $T - 1$ positions (excluding position t^*):

$$\sum_{t'=1, t' \neq t^*}^T |s_{t,t'}^{(\ell)} - s'_{t,t'}^{(\ell)}| \|x_{t'}^{(\ell)}\| \leq \|X^{(\ell)}\|_{2,\infty} \sum_{t'=1, t' \neq t^*}^T O(1/T^2) = O(1/T).$$

■

- j) *Proof.* Combine the decomposition from part (e) with the bounds from parts (f) and (i). We prove by induction on ℓ that

$$\|x_t^{(\ell)} - x_t'^{(\ell)}\| = \begin{cases} O(1), & t = t^*, \\ O(1/T), & t \neq t^*. \end{cases}$$

For $t \neq t^*$: The difficult part is $O(1/T)$ by part (i). The easy part $\sum_{t'=1}^T s'_{t,t'}(\ell) \|x_{t'}^{(\ell)} - x_t^{(\ell)}\|$ satisfies

$$\sum_{t'=1}^T s'_{t,t'}(\ell) \|x_{t'}^{(\ell)} - x_t^{(\ell)}\| \leq \sum_{t'=1}^T s'_{t,t'}(\ell) \cdot O(1/T) = O(1/T),$$

where the bound $O(1/T)$ follows from the inductive hypothesis for all t' (including $t' = t^*$). Hence $\Delta_t^{(\ell)} = O(1/T) + O(1/T) = O(1/T)$.

For $t = t^*$: The difficult part remains $O(1/T)$ by part (i), but the easy part now includes the large term at $t' = t^*$:

$$s'_{t^*,t^*}(\ell) \|x_{t^*}^{(\ell)} - x_{t^*}^{(\ell)}\| = O(1) \cdot O(1) = O(1).$$

Thus $\Delta_{t^*}^{(\ell)} = O(1/T) + O(1) = O(1)$.

By the inductive step, $\|x_t^{(\ell+1)} - x_t'^{(\ell+1)}\| = \|f(z_t^{(\ell)}, x_t^{(\ell)}) - f(z_t'^{(\ell)}, x_t'^{(\ell)})\|$ inherits the above bound structure through the Lipschitz property of f .

In particular, since the eos position is $T > t^*$, we have

$$\|h(y) - h(y')\| = \|x_T^{(L)} - x_T'^{(L)}\| = O(1/T).$$

■

k) *Proof.* If $y \in \text{Odd-1}$ and we flip one bit, then the parity changes, so y' is not in Odd-1 . The final representation changes by only $O(1/T)$, hence the logit $h(y)^\top w$ changes by at most $O(1/T)$ as well. As $|y| \rightarrow \infty$, this vanishes, so the classifier cannot maintain a uniform margin between the two classes for arbitrarily long strings. ■

l) *Proof.* With layer normalization, the argument remains valid as long as the normalization map is Lipschitz on the relevant domain. If the variance lower bound is strictly positive, this is the case. If $\varepsilon = 0$, however, the Lipschitz constant can blow up when the variance becomes small, and the proof no longer goes through uniformly.

The result is asymptotic: it shows that sufficiently long strings become hard to distinguish by this transformer family. For any fixed maximal length T^* , the model may still classify strings correctly on that finite range; the impossibility is about uniform recognition for unbounded length. ■

✓ LLM Assignment 1: Representation Surgery (Coding)

IMPORTANT: MAKE A COPY OF THIS NOTEBOOK AND WORK IN THERE.

Welcome to the coding part of the Representation Surgery question in Assignment 1. The goal of this portion is to give you a deeper intuition for how representation surgery works and its strengths/limitations via hands-on implementations.

It consists of 2 parts:

1. Implementing linear concept erasure as described in the assignment, and applying it to a synthetic dataset.
2. Applying concept erasure to erase the concept of "gender" from the BERT representations of a dataset of biographies.

In this assignment, you'll have a series of tasks. For each task you will need to write some code and produce plots and/or accuracy results. You will need to export the notebook as a pdf and attach it to your submission. Please make sure all deliverables for each task (e.g., plots and accuracy results) are clearly visible, as we will be grading you on these outputs.

```
!git clone https://github.com/rycolab/llm-representation-surgery.git
```

```
fatal: destination path 'llm-representation-surgery' already exists and is not an empty directory.
```

```
# These are all the imports you should need
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.datasets import make_classification
from sklearn.cross_decomposition import CCA
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import LabelEncoder
from typing import Tuple
np.random.seed(42)
```

✓ 0. Constructing a synthetic dataset

First, we'll give you a synthetic dataset encoding a concept, i.e., the target labels.

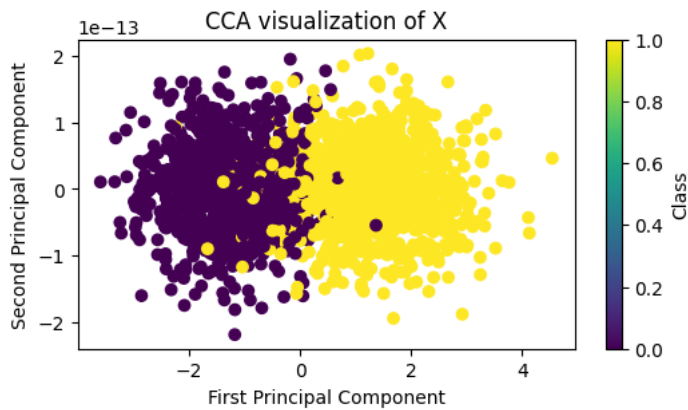
```
n, d, k = 2048, 128, 2
X, Z = make_classification(
    n_samples=n,
    n_informative=k,
    n_features=d,
    n_classes=k,
    random_state=1,
)
print(f"Shape of X: {X.shape}, Shape of Z: {Z.shape}")
```

```
Shape of X: (2048, 128), Shape of Z: (2048,)
```

What does this dataset look like? Visualizing 128-dimensional data is hard, so we'll reduce the dimensionality to 2 using CCA and plot the result.

```
def plot_cca(X, Z):
    """
    Use Canonical Component Analysis to visualize the dataset X and label Z.
    """
    Z_one_hot = np.zeros((len(Z), len(set(Z))))
    Z_one_hot[np.arange(len(Z)), Z] = 1
    cca = CCA(n_components=2)
    X_cca = cca.fit_transform(X, y=Z_one_hot)[0]
    plt.figure(figsize=(6, 3))
    plt.scatter(X_cca[:, 0], X_cca[:, 1], c=Z, cmap='viridis')
    plt.colorbar(label='Class')
    plt.xlabel('First Principal Component')
    plt.ylabel('Second Principal Component')
    plt.title('CCA visualization of X')
    plt.show()

plot_cca(X, Z)
```



As you can see, in these two components, the two concepts are fairly separable.

- Task 0.1: Implement a logistic regression classifier predict the concept Z from X .

FOR THE SUBMISSION: Print the accuracy your classifier scores on the data.

```
def logreg(X: np.ndarray, Z: np.ndarray) -> float:
    """
    Train a logistic regression classifier to predict concept Z from X.

    Args:
        X - the data
        Z - the concept/label

    Return:
        Accuracy in predicting Z from X.
    """
    # TODO:
    clf = LogisticRegression(max_iter=1000, random_state=42)
    clf.fit(X, Z)
    return clf.score(X, Z)

print(f"Accuracy before erasure: {logreg(X, Z):.3f}")
```

Accuracy before erasure: 0.941

- 1. Implementing linear concept erasure

- Task 1.1: Validate that the resulting whitening and unwhitening matrices satisfy their definition and goals.

That is, verify (a) $W W^T = I$ and (b) $cov(XW) = I$.

FOR THE SUBMISSION: Include your plots showing the whitening matrices.

```
def compute_W(X: np.ndarray) -> Tuple[np.ndarray, np.ndarray]:
    """
    Compute the whitening and unwhitening matrix for the data using the ZCA whitening method
    (see Appendix A of https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf for more details).
    We give this to you as a numerically stable implementation.
    """
    sigma = np.cov(X.T)

    L, V = np.linalg.eigh(sigma)

    # Assuming PSD; account for numerical error
    L = L.clip(0.0)

    # Threshold used by torch.linalg.pinv
    mask = L > (L[-1] * sigma.shape[-1] * np.finfo(float).eps)

    with np.errstate(divide='ignore'):
        W = V * np.where(mask, 1/np.sqrt(L), 0.0) @ V.T
        W_inv = V * np.where(mask, np.sqrt(L), 0.0) @ V.T

    return W, W_inv

def heatmap(arr, title):
    plt.figure(figsize=(10, 8))
```

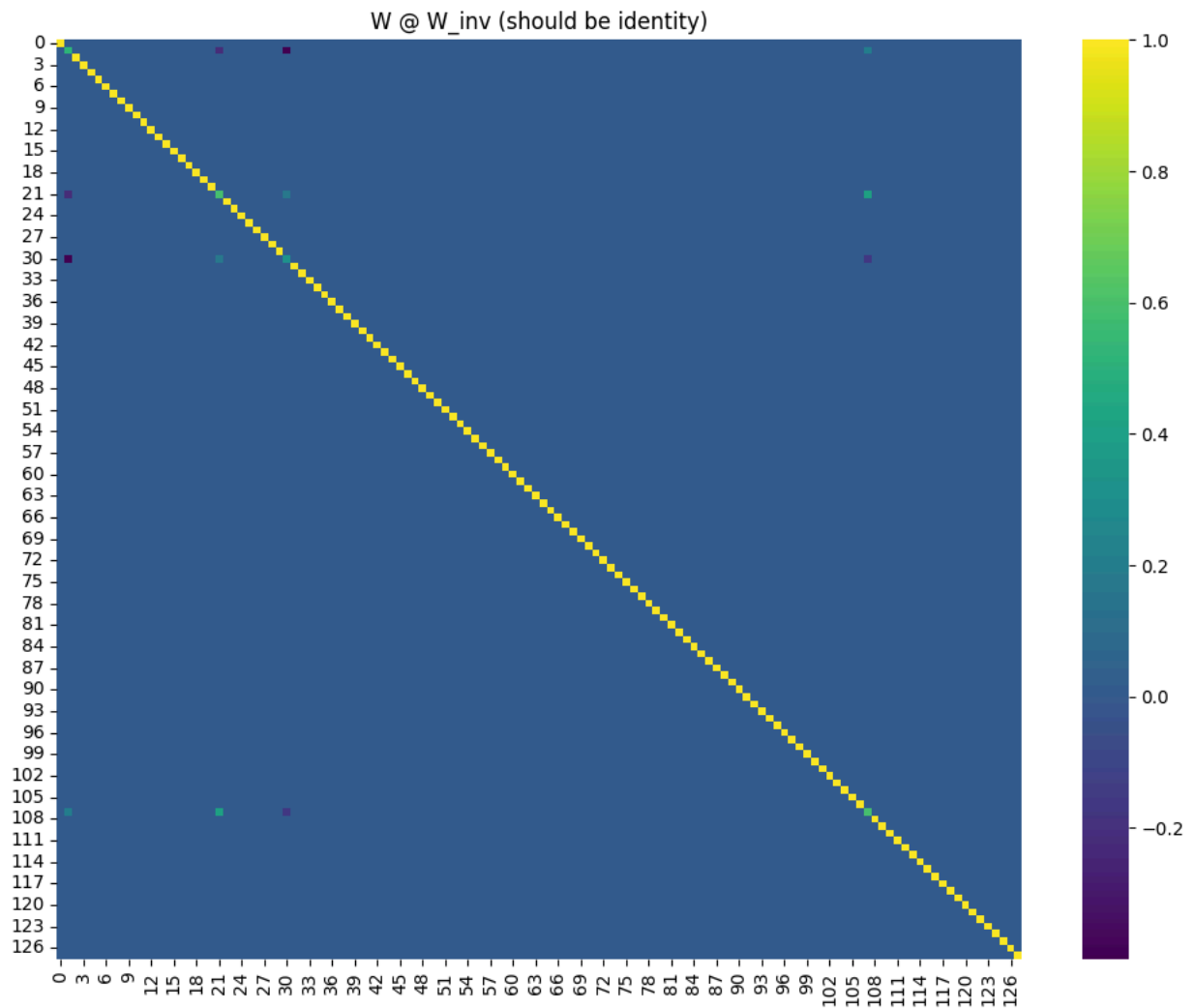


```
sns.heatmap(arr, cmap='viridis')
plt.title(title)
plt.tight_layout()
plt.show()
```

```
W, W_inv = compute_W(X)
```

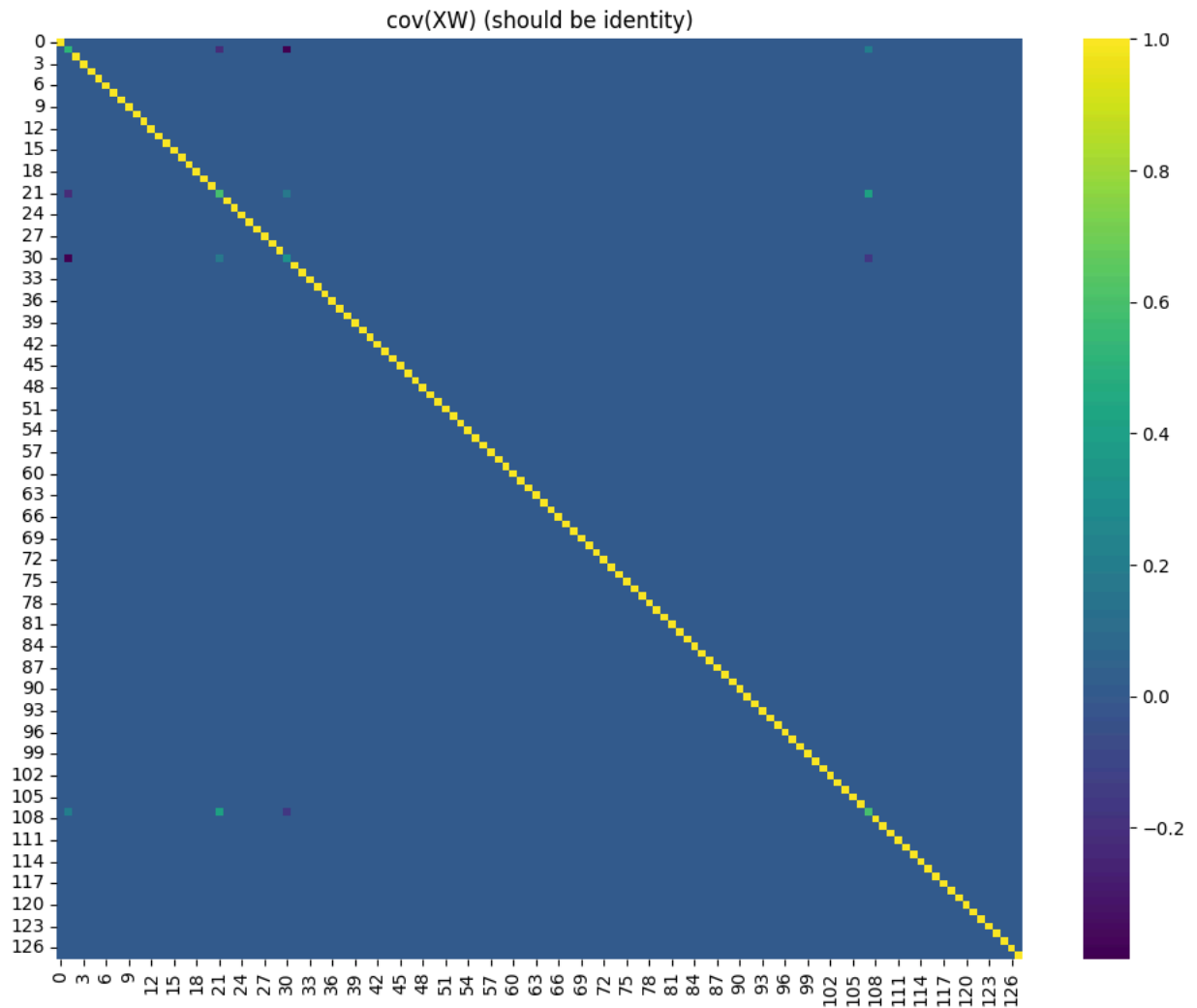
```
def verify_W_W_inv(W, W_inv):
    """
    Plot a heatmap of  $W @ W\_inv$  to verify it satisfies the definition of whitening.
    """
    # TODO
    result = W @ W_inv
    heatmap(result, 'W @ W_inv (should be identity)')

verify_W_W_inv(W, W_inv)
```



```
def verify_cov_XW(X, W):
    """
    Plot a heatmap of the covariance of XW to verify it satisfies the definition of whitening.
    """
    # TODO
    X_whitened = X @ W
    cov_XW = np.cov(X_whitened.T)
    heatmap(cov_XW, 'cov(XW) (should be identity)')

verify_cov_XW(X, W)
```



Task 1.2: Implement linear concept erasure

Use the equations derived from the written part of the assignment.

FOR THE WRITEUP: Include your plot and accuracy of the embeddings before erasure and after erasure.

```
def erase(X: np.ndarray, Z: np.ndarray) -> np.ndarray:
    """
    Implement linear concept erasure.

    Args:
        X - the data, shape (n, d)
        Z - the concept/label, shape (n,)

    Return:
        The data with the concept erased. Shape (n, d).
    """
    # TODO:
    # Compute mean
    mu = X.mean(axis=0)
    X_centered = X - mu

    # Compute whitening matrix and its pseudoinverse
    W, W_inv = compute_W(X)
    W_pinv = np.linalg.pinv(W)

    # Create one-hot encoding of Z
    Z_unique = np.unique(Z)
    Z_one_hot = np.zeros((len(Z), len(Z_unique)))
    for i, z_val in enumerate(Z_unique):
        mask = Z == z_val
        Z_one_hot[mask, i] = 1

    # Center Z
    Z_centered = Z_one_hot - Z_one_hot.mean(axis=0)

    # Compute covariances
    Sigma_XZ = (X_centered.T @ Z_centered) / len(X) # cross-covariance
```

```

# Compute the matrix  $W\Sigma_{\{X,Z\}}$ 
WS = W @ Sigma_XZ # shape (d, k) where k = # of classes

# Compute projection:  $\Pi = W\Sigma_{\{X,Z\}} (W\Sigma_{\{X,Z\}})^{-1}$ 
WS_pinv = np.linalg.pinv(WS)
Pi = WS @ WS_pinv # shape (d, d)

# Apply the transformation:  $r(x) = x - W^+ \Pi W (x - \mu)$ 
X_erased = X - (X_centered @ W.T @ Pi.T @ W_pinv.T)

return X_erased

erased_X = erase(X, Z)

```

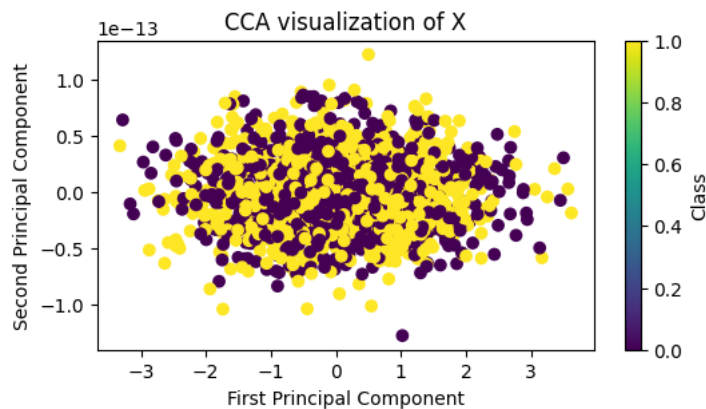
Now, let's validate that the concept has been erased.

```

plot_cca(erased_X, Z)
print(f"Accuracy after erasure: {logreg(erased_X, Z)}")

```

/usr/local/lib/python3.12/dist-packages/sklearn/cross_decomposition/_pls.py:113: ConvergenceWarning: Maximum number of iterations reached. warnings.warn("Maximum number of iterations reached", ConvergenceWarning)



2. Erasing gender bias

In this portion, we'll apply the same method to a real dataset of word embeddings of biography data. Your goal is to (a) construct sentence embeddings and (b) erase the gender concept from these embeddings. Use the `hard_text_untokenized` column for your embeddings.

```

df = pd.read_csv("llm-representation-surgery/data/biographies.csv")
df.head()

```

	g	p	text	start	hard_text	hard_text_untokenized	text_without_gender
0	f	psychologist	Brandi Smith is a registered psychologist in t...	139	Ms. Smith is experienced in providing psychoth...	Ms. Smith is experienced in providing psychoth...	_ _ is experienced in providing psychotherapy...
1	m	software_engineer	Duarte Nunes is a Software Engineer working on...	56	He has a background in concurrent programming ...	He has a background in concurrent programming,...	_ has a background in concurrent programming, ...
2	f	software_engineer	Lauren Ellsworth is a Senior Software Engineer...	139	In her years of development , Lauren has contri...	In her years of development, Lauren has contri...	In _ years of development, _ has contributed t...

Nächste Schritte: [Code mit df generieren](#) [New interactive sheet](#)

Task 2.1: Compute embeddings and use the `erase` function defined above to erase the gender concept from the embeddings.

FOR THE FOR THE SUBMISSION: Include: (a) CCA plot of the embeddings before erasure, (b) CCA plot of the embeddings after erasure, (c) logistic regression accuracy before erasure, and (d) logistic regression accuracy after erasure.

```

def extract_gender_labels(df: pd.DataFrame) -> np.ndarray:

```

```

"""
Extract the gender labels from the dataframe.
"""
# TODO:
gender = df['g'].values
le = LabelEncoder()
gender_labels = le.fit_transform(gender)
return gender_labels

def compute_embeddings(df: pd.DataFrame) -> np.ndarray:
    """
    Compute the embeddings using a pretrained model for the "hard_text_untokenized" column of the given dataframe
    Feel free to experiment with different models; a good starting point is the classic BERT model, "google-bert/"
    Also feel free to use any libraries which would be helpful, such as `transformers` or `sentence_transformers`
    """
    # TODO:
    try:
        from sentence_transformers import SentenceTransformer
        model = SentenceTransformer("all-MiniLM-L6-v2")
        texts = df['hard_text_untokenized'].values
        embeddings = model.encode(texts, show_progress_bar=True)
        return np.array(embeddings)
    except:
        # Fallback: use transformers library
        from transformers import AutoTokenizer, AutoModel
        import torch

        model_name = "google-bert/bert-base-uncased"
        tokenizer = AutoTokenizer.from_pretrained(model_name)
        model = AutoModel.from_pretrained(model_name)

        texts = df['hard_text_untokenized'].values
        embeddings = []

        for text in texts:
            inputs = tokenizer(text, return_tensors="pt", truncation=True, max_length=512)
            with torch.no_grad():
                outputs = model(**inputs)
            # Use [CLS] token embedding (first token)
            embeddings.append(outputs.last_hidden_state[0, 0, :].numpy())

        return np.array(embeddings)

def erase_gender(embeddings: np.ndarray, gender_labels: np.ndarray) -> np.ndarray:
    """
    This function should do 3 things:
    1. Erase the gender concept from the embeddings and return these new embeddings.
    2. Visualize the embeddings before erasure and print the accuracy before erasure.
    3. Visualize the embeddings after erasure and print the accuracy after erasure.

    Args:
        embeddings: The embeddings to erase the gender concept from.
        gender_labels: The gender labels for the embeddings.

    Returns:
        The embeddings with the gender concept erased.
    """
    # TODO:
    print("Before erasure:")
    plot_cca(embeddings, gender_labels)
    acc_before = logreg(embeddings, gender_labels)
    print(f"Accuracy before erasure: {acc_before:.3f}")

    # Apply erasure
    embeddings_erased = erase(embeddings, gender_labels)

    # After erasure
    print("\nAfter erasure:")
    plot_cca(embeddings_erased, gender_labels)
    acc_after = logreg(embeddings_erased, gender_labels)
    print(f"Accuracy after erasure: {acc_after:.3f}")

    return embeddings_erased

gender_labels = extract_gender_labels(df)
embeddings = compute_embeddings(df)
embeddings_erased = erase_gender(embeddings, gender_labels)

```

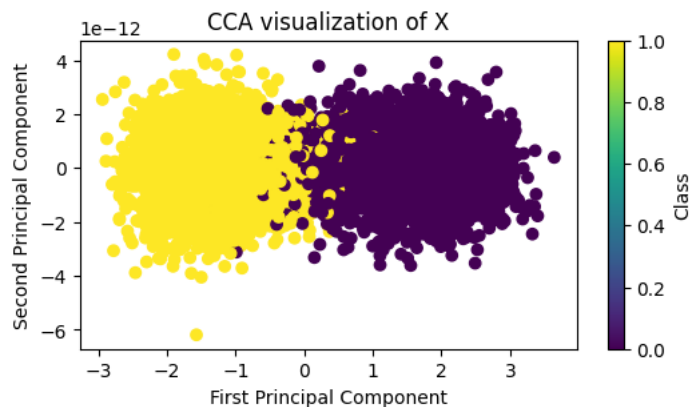
```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/t)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(
modules.json: 100% 349/349 [00:00<00:00, 35.3kB/s]
config_sentence_transformers.json: 100% 116/116 [00:00<00:00, 11.4kB/s]
README.md: 10.5k/? [00:00<00:00, 463kB/s]
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limit
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a
sentence_bert_config.json: 100% 53.0/53.0 [00:00<00:00, 3.66kB/s]
config.json: 100% 612/612 [00:00<00:00, 63.1kB/s]
model.safetensors: 100% 90.9M/90.9M [00:01<00:00, 208MB/s]
Loading weights: 100% 103/103 [00:00<00:00, 409.10it/s, Materializing param=pooler.dense.weight]
BertModel LOAD REPORT from: sentence-transformers/all-MiniLM-L6-v2
Key | Status | |
-----+-----+
embeddings.position_ids | UNEXPECTED | |

Notes:
- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch
tokenizer_config.json: 100% 350/350 [00:00<00:00, 32.2kB/s]
vocab.txt: 232k/? [00:00<00:00, 9.05MB/s]
tokenizer.json: 466k/? [00:00<00:00, 24.7MB/s]
special_tokens_map.json: 100% 112/112 [00:00<00:00, 13.1kB/s]
config.json: 100% 190/190 [00:00<00:00, 21.9kB/s]
Batches: 100% 313/313 [00:11<00:00, 62.27it/s]

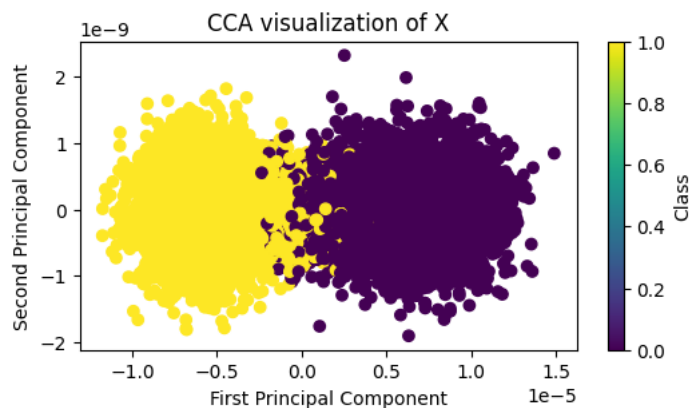
```

Before erasure:



Accuracy before erasure: 0.985

After erasure:



Accuracy after erasure: 0.547

✓ Task 2.2: Verifying minimally-destructive edits.

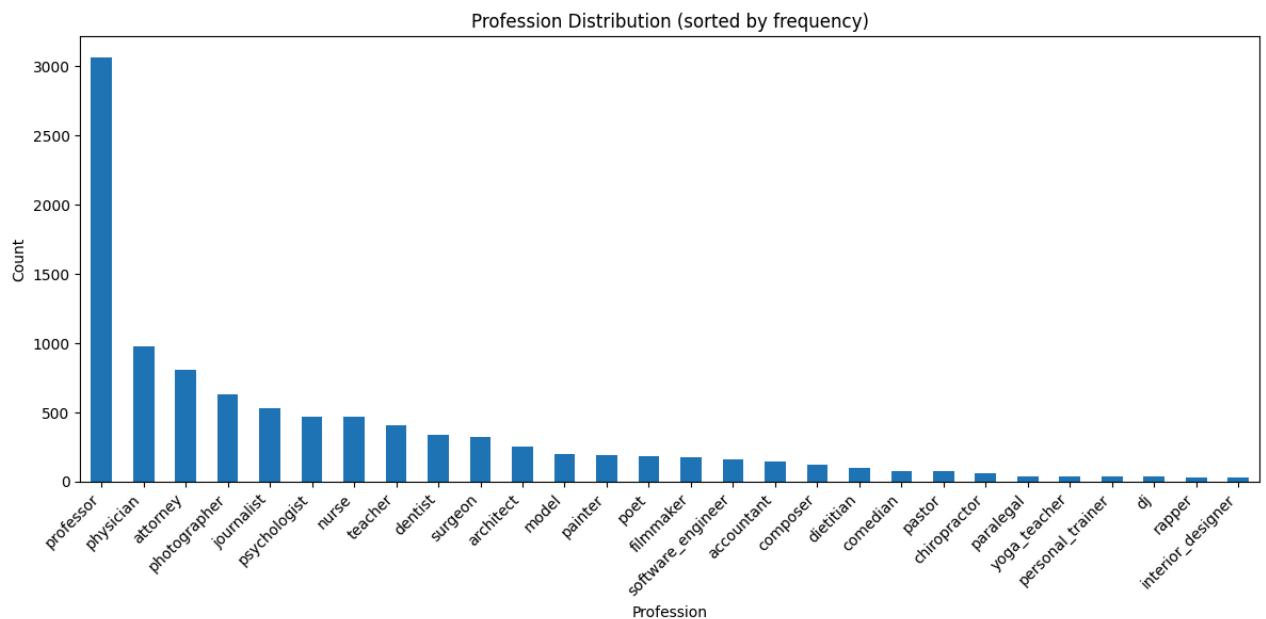
One key aspect of concept erasure is we want the edits to be *minimal*, i.e., they should not destroy other aspects of the representations. To test the degree to which the representation is preserved, use the profession as a proxy. Show that the profession

information is preserved within the embeddings by computing the accuracy (a) before erasure and (b) after erasure.

FOR THE SUBMISSION: Report the accuracy before and after erasure.

```
def visualize_profession(df: pd.DataFrame) -> np.ndarray:
    """
    Visualize the label distribution of professions, sorted from highest to lowest.
    """
    # TODO:
    profession_counts = df['p'].value_counts().sort_values(ascending=False)
    plt.figure(figsize=(12, 6))
    profession_counts.plot(kind='bar')
    plt.title('Profession Distribution (sorted by frequency)')
    plt.xlabel('Profession')
    plt.ylabel('Count')
    plt.xticks(rotation=45, ha='right')
    plt.tight_layout()
    plt.show()

visualize_profession(df)
```



```
def verify_profession_intact(embeddings: np.ndarray, embeddings_erased: np.ndarray, df: pd.DataFrame) -> np.ndarray:
    """
    Verify the profession information is preserved within the embeddings.
    """
    # TODO:
    professions = df['p'].values
    le = LabelEncoder()
    profession_labels = le.fit_transform(professions)

    # Accuracy before erasure
    acc_before = logreg(embeddings, profession_labels)
    print(f"Profession prediction accuracy BEFORE erasure: {acc_before:.3f}")

    # Accuracy after erasure
    acc_after = logreg(embeddings_erased, profession_labels)
    print(f"Profession prediction accuracy AFTER erasure: {acc_after:.3f}")

    print(f"Profession information preservation: {acc_after/acc_before*100:.1f}%")

verify_profession_intact(embeddings, embeddings_erased, df)
```

```
Profession prediction accuracy BEFORE erasure: 0.784
Profession prediction accuracy AFTER erasure: 0.779
Profession information preservation: 99.4%
```

- Task 2.3: Show that using a nonlinear classifier, you can recover the concept from the erased embeddings.

While we've successfully removed gender from being *linearly encoded* in the embeddings, they can still be extracted using other models. Your goal is to extract the concept using some other model.

FOR THE SUBMISSION: Report the accuracy before and after erasure.

```
def nonlinear_classifier(embeddings: np.ndarray, gender_labels: np.ndarray) -> float:
    """
    Train a nonlinear classifier on the embeddings to predict the gender labels and return the accuracy.
    """
    # TODO
    try:
        # Try using sklearn's MLPClassifier (Neural Network)
        from sklearn.neural_network import MLPClassifier
        from sklearn.preprocessing import StandardScaler

        # Standardize embeddings before passing to neural network
        scaler = StandardScaler()
        embeddings_scaled = scaler.fit_transform(embeddings)

        clf = MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=500, random_state=42)
        clf.fit(embeddings_scaled, gender_labels)
        return clf.score(embeddings_scaled, gender_labels)
    except:
        # Fallback: use RandomForest
        from sklearn.ensemble import RandomForestClassifier
        clf = RandomForestClassifier(n_estimators=100, random_state=42)
        clf.fit(embeddings, gender_labels)
        return clf.score(embeddings, gender_labels)

# Test nonlinear classifier on both original and erased embeddings
print("Nonlinear Classifier Results:")
print(f"Accuracy on original embeddings: {nonlinear_classifier(embeddings, gender_labels):.3f}")
print(f"Accuracy on erased embeddings: {nonlinear_classifier(embeddings_erased, gender_labels):.3f}")
```

```
Nonlinear Classifier Results:
Accuracy on original embeddings: 1.000
Accuracy on erased embeddings: 1.000
```